

LEADERSHIP WITH DATA

Agile for Data Leaders, Not Scrum Masters

How to make data work predictable, protect roadmaps from ad-Hoc, and rebuild trust with the business



JIMMY PANG

DEC 22, 2025



13



1



1

Share



A simple illustration of the steps of Agile

Data teams rarely fail because of SQL or Python. They fail because work is chaotic, priorities change mid-week, and “simple” requests disappear into a black box, eroding trust with the business. For

© 2026 Jimmy Pang · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture

Tech and Product have largely solved this with Agile, but most data teams copy the ceremonies without adapting them to the reality of ad-hoc questions, incidents, and constantly shifting stakeholders.

This article shows how Scrum and Kanban can actually work for BI, analytics engineering, data engineering, and data science: how to structure the work, protect focus, and still respond to urgent questions. Treat it as a menu, not a religion—take what fits, ignore what does not, and evolve your own way of working over time.

Why Agile for Data Teams (Really)?

For a data leader, the real problem is not effort but predictability: no one can say when anything will be done, and every fire drill blows up the roadmap. Stakeholders feel this as missed deadlines, half-finished dashboards, and models that never quite make it to production, even though the team is visibly busy.

Agile gives data teams a simple way to make work visible, limit how many things are in progress, and negotiate trade-offs with the business instead of just saying “we’re overloaded”.

Scrum and Kanban are just two patterns to solve slightly different problems. Kanban shines when the team lives in interrupts—ad-hoc analysis, one-off data pulls, small dashboard tweaks, and operational data issues. Scrum fits better when the work looks like products and platforms—rebuilding a data mart, rolling out a metrics layer, or industrialising an experimentation framework where you need focus and stable sprints.

Most real data teams sit somewhere in between. The rest of this article focuses on designing a way of working that matches your mix of BI, AE, DE, and DS, instead of pretending your world is as tidy

as pure software development.

For data leaders and executives, this reframes Agile from “engineering ceremony” to a governance tool: a way to make data work predictable, negotiable, and transparently trade-off against other priorities.

Agile in a Data Context (BI, AE, DS, DE)

Agile only works if BI, AE, DE, and DS each use a way of working that matches the kind of work and uncertainty they face.

- **Business Intelligence (BI / analysts)**. Highly reactive and business-facing: ad-hoc questions, KPI definitions, dashboard decision support. Kanban with strict WIP limits, clear triage rules, and short planning horizons usually works best so analysts can re-prioritize without blowing up longer projects
- **Analytics Engineering (AE)**. Product-like work: modeling layer dbt projects, data marts, contract changes that affect many downstream users. Scrum or hybrid Scrumban with 1–2 week sprints, clear definitions of done, and ceremonies that surfacing breaking changes before they hit production tends to fit well
- **Data Engineering (DE)**. Pipelines, ingestion, orchestration, a platform work mix roadmap initiatives with operational incidents. A dual-track setup is effective: Scrum for planned roadmap work (new pipelines, migrations) and Kanban for on-call and incident response, with explicit capacity reserved for each
- **Data Science (DS)**. High-uncertainty experimentation and model development rarely map cleanly to fixed-scope tickets. Longer “discovery sprints” focused on hypotheses and decision points plus Kanban-style tracking of experiments and model iteration

usually give the right balance of structure and flexibility.

For data leaders and executives, this means designing operating modes per sub-team, aligned to their risk profile and stakeholders, while still rolling everything up into one coherent portfolio.

Scrum vs Kanban for Data Work

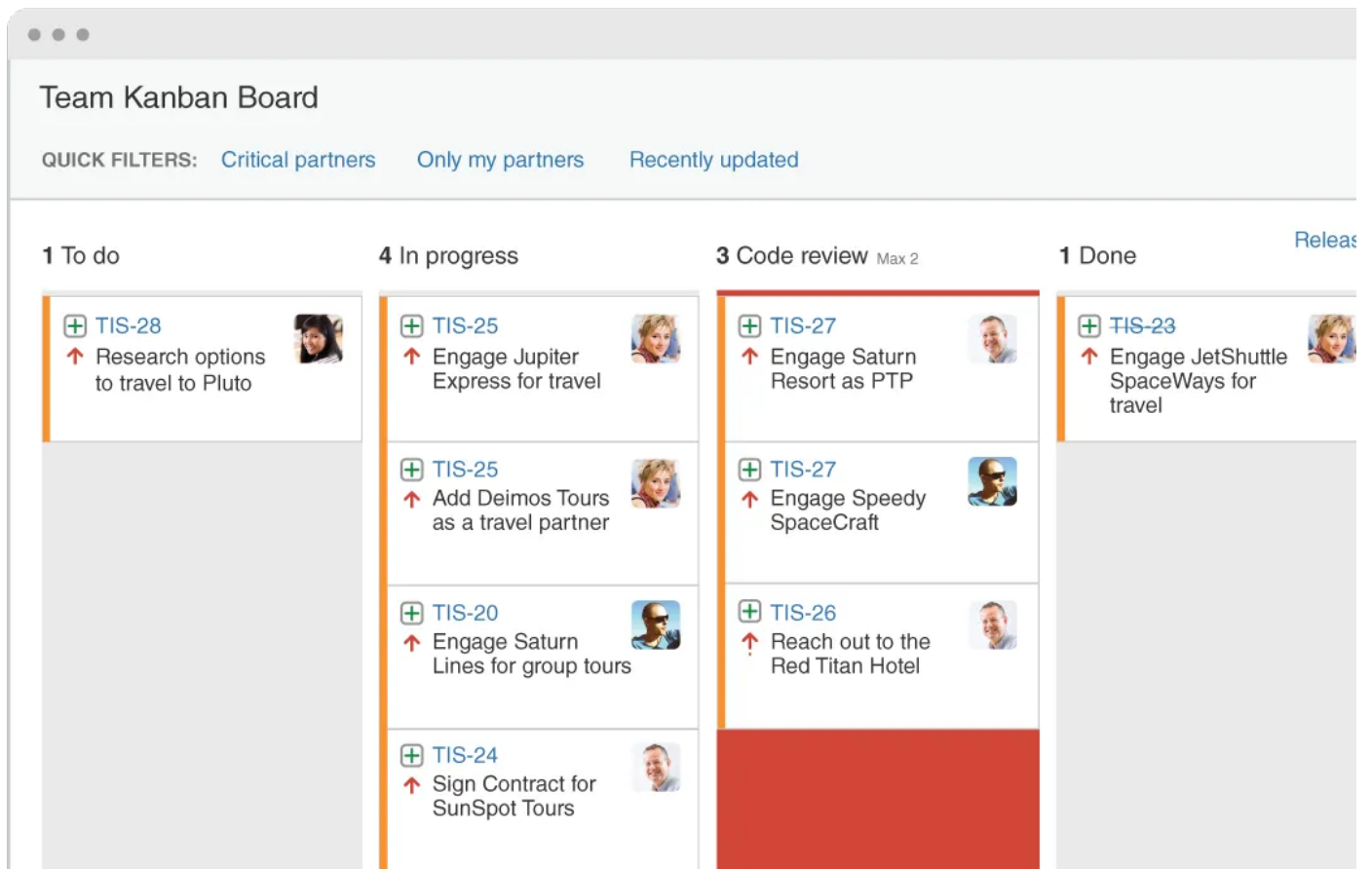
Choose Scrum, Kanban, or a hybrid based on whether your main problem is big projects needing focus or a stream of small, interrupt-driven tasks.

	Scrum	Kanban
Origin	Software development	Lean manufacturing
Ideology	Learn through experiences, self-organize and prioritize, and reflect on wins and losses to continuously improve.	Use visuals to improve work-in-progress
Cadence	Regular, fixed-length sprints (i.e. two weeks)	Continuous flow
Practices	Sprint planning, sprint, daily scrum, sprint review, sprint	Visualize the flow of work, limit work-in-progress, manage flow, incorporate

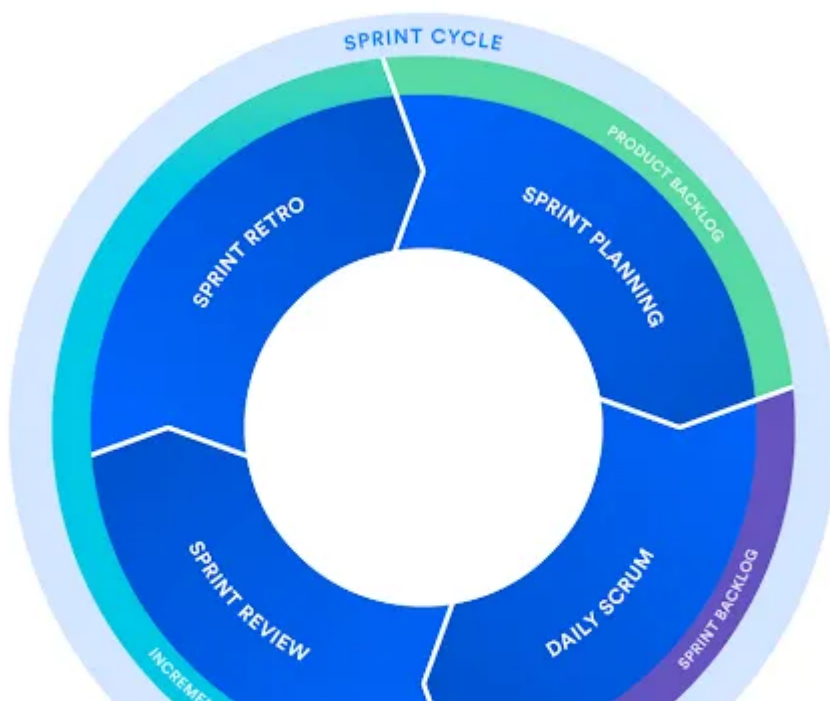
	retrospective	new, incorporate feedback loops
Roles	Product owner, scrum master, development team	No required roles

From [Atlassian: Kanban vs Scrum](#)

- **When Kanban shines.** Best for continuous, relatively small, and highly reactive work: ad-hoc analyses, one-off data pulls, dashboard tweaks, small semantic-layer changes, and minor data-quality fixes. A Kanban board makes all requests visible, limits work-in-progress so analysts are not juggling ten tickets, and makes it easy to re-prioritize as the business changes its mind—ideal for BI and support-heavy DE teams.
- **When Scrum shines.** Best for project- or product-like work where partial delivery is risky and coordination matters: rebuilding a core data mart, re-platforming pipelines, rolling out a metrics layer, or industrialising an ML model. One- to two-week sprints create a contract with stakeholders, make capacity explicit, protect the team from endless “quick questions” that derail deeper work.
- **What most data teams actually need.** In practice, many teams run a hybrid: a Scrum-style sprint cadence for roadmap work, plus a Kanban lane for incidents and truly urgent requests with a capped share of capacity each sprint. This preserves predictability for strategic initiatives while admitting that data teams live closer to the business’ chaos than most software squads. Done well, this hybrid reduces the blast radius of incidents while keeping strategic rebuilds and platform work moving, so big bets do not stall every time sales or operations hits a rough week.



A classic Kanban board from [Atlassian](#)





Sprint Cycle in Scrum

For senior stakeholders, this gives you a simple lens: Kanban when you care about fast response to changing questions, Scrum when you care about reliable delivery of big bets—most real teams need a deliberate mix of both.

Essential Ceremonies for Data Teams

4 lightweight rituals – Standup, Grooming, Planning, and Retro – are enough to keep a data team aligned without drowning it in meetings. They shorten the loop from question to decision, reduce the number of “surprise” delays, and make it easier to see which initiatives are actually moving the needle.

Daily Standup: Walk the Board or Classic Huddle

Daily standup is a 10–15 minute status check to keep the sprint (or Kanban flow) on track, not a problem-solving workshop. Two simple formats work well for data teams:

- **Walk-the-board.** Move quickly through tickets in progress (from “In Review” back to “To Do”); for each, the owner says what changed, whether it is blocked, and if it needs re-prioritisation or parking.
- **Classic huddle (no board).** Each person shares what they finished yesterday, what they are doing today, and any blockers or help needed.

Both formats exist to surface stuck work and decide if anything needs to move, not to host long discussions. Keep small talk (“how was your weekend?”) outside the standup to avoid turning a 10-minute check-in into a 30-minute meeting.

For managers and stakeholders, this is how the team gets early warning on blocked work and slipping commitments in 10 minutes a day, instead of finding out at the end of the sprint.

Grooming: Protecting Quality Before Work Starts

Grooming is where the team turns messy ideas into work the squad can actually commit to. The goals are simple: make sure everyone understands the context, slice work small enough to finish within a sprint, and align on a shared notion of effort.

In practice, review new tickets with domain experts when possible to clarify the “why”, business impact, and expected outputs (dashboards, tables, models, experiments), and turn vague asks into concrete specs with examples and acceptance criteria. If a ticket feels too big or fuzzy, break it into smaller, testable tasks (for example, “align KPI definition”, “create intermediate model”, “build dashboard slice for Ops only”) and link them so dependencies are explicit.

For story-point estimation, keep it simple and use grooming to calibrate the team. Use a small scale such as 0, 1, 2, 3, 5, 8, where points reflect effort and complexity, not exact hours, and anchor it with concrete examples:

0: up to 30 minutes (often no ticket needed).

1: 30 minutes–2 hours.

2: 2–4 hours.

3: 4 hours–1 day.

5: 1–2 days.

8: 2–3 days.

13: 3–5 days.

Most healthy tickets should land around 3–5 points; if something looks like an 8, treat it as a signal to split it into at least 1 parts, for example, a 3-point discovery or spike and a 5-point implementation task. Anything above 13 almost always hides multiple pieces of work and should be broken down before it can enter a sprint. Calibration happens by comparing estimates out loud: if a senior feels a ticket is 3 points but a junior thinks it is 5, agree to log it as 5 so the number represents effort for the team, not the fastest person. Routine micro-tasks such as adding a user to a Tableau group can either be 0-point items or handled via simple runbooks without cluttering the main backlog.

For data leaders, strong grooming is where you actually buy predictability: vague asks become clear scope, effort is calibrated across the team, and surprises later in the sprint go down.

Planning: Turning Points into a Sprint Commitment

Sprint planning turns groomed tickets into a realistic commitment for the next iteration. Estimation happens in grooming; planning about how much work fits, which tickets to take, and what trade-offs to make with stakeholders.

A practical approach for data teams:

- **Start with actual capacity, not headcount.** Subtract public holidays, vacations, and major absences from the sprint calendar so you plan with a realistic view of who is available.
- **Have each person propose their own mini-plan.** Every team member drafts a slice of the sprint: one primary deliverable (for example, “ship v1 of the new Ops performance dashboard”) plus supporting tickets. The squad then challenges scope and over-commitment so planning is a collaborative negotiation, not a top-down assignment.
- **Under-commit and over-deliver.** Choose fewer, higher-value tickets than the theoretical maximum based on past velocity; the team finishes early, pull the next most important items from the backlog.

For more mature teams, add a short mid-sprint check-in (besides daily standup) to check whether the sprint goal still holds, and maintain a “Next+1” placeholder sprint to park lower-priority tickets and give stakeholders a realistic horizon without over-promising the current sprint.

For you as a CDO/VP, this is the mechanism that turns “we’ll try our best” into a clear contract: what the team will deliver, what happens when priorities change, and how often that contract is broken.

During the Sprint: Protecting Focus

Once the sprint starts, treat the plan as a contract, not a wishlist: the default is to finish what was committed, not to reshuffle tickets every few days. Avoid “intruder tickets” unless there is a production incident, a serious data-quality issue, or

all planned work is blocked; if something must enter mid-sprint, ticket of similar size should leave.

When engineers or analysts finish early, encourage them to help with reviews, documentation, and unblocking peers before pulling new work, or to prepare for the next sprint by drafting and refining tickets so upcoming work is easier to start. This reduces context-switching, accelerates overall throughput, and avoids the pattern where visible individuals absorb more ad-hoc work while roadmap items quietly slip.

At Sprint End: Handling Spillover

As the sprint ends, do everything possible to avoid spillover so the sprint closes cleanly. Any unfinished ticket should not quietly remain "In Progress": carry it explicitly into the next sprint, re-estimate if needed, and re-prioritize it against new work. Spillover is more than an administrative nuisance; habitual spillover signals failed expectations, makes planning harder, and erodes stakeholders' trust in the data team's commitments, so repeated spillover should become a retro topic to adjust estimation, scoping, or how aggressively the team commits.

Retrospective: Fixing the System, Not the People

A Retrospective is the team's feedback loop: one focused hour at the end of each sprint to improve how work gets done, not to blame individuals. The aim is to spot patterns that hurt delivery and trust and to agree on 1-2 concrete changes for the next sprint.

Make sure the basics are in place: psychological safety, a clear norm that feedback is about the system and work, and a shared understanding that the goal is to make life easier for everyone.

Look for recurring themes such as late-breaking requests from the same stakeholder group, metric definition disputes that slow every dashboard, or the same upstream system repeatedly breaking pipelines.

A simple structure works well for data teams: write, group, vote, solve. Give everyone 5–10 minutes to silently add cards for “What went well” and “What went less well / should change”, then quickly group similar cards, vote on which clusters matter most, and discuss the top ones to agree on 1–2 specific experiments or process changes for the next sprint.

Tools like [EasyRetro](#) and [RetroTool](#) make this flow easy to run remotely and asynchronously. Always document agreed actions in a shared space (Confluence, Notion) and review them briefly at the next retro so improvements compound over time. Many high-performing teams also start retros by quickly reviewing their sprint report (completed vs committed, spillover, intruder tickets) to ground the conversation in observable data rather than memory.

For senior leadership, retros are your cheapest continuous-improvement engine: every sprint you get one or two concrete changes that reduce incidents, noise, and stakeholder escalations without a re-org or new tool.

Handling Ad-Hoc Requests Without Chaos

Ad-hoc will never go away, but clear intake rules, lanes, and trade-offs stop it from silently destroying your roadmap.

1. Standardize how requests enter

Use a single intake channel (form, Jira/Linear issue type, or

dedicated Slack channel) with mandatory fields: business question it supports, decision it supports, deadline, and impact if it is not done. Push back on drive-by asks that bypass this intake and route everything through the same queue so prioritization is transparent and searchable later; this filters low-value noise and forces requesters to think about what they actually need.

Over the past few years, a more structured framework for scalable stakeholder management has worked extremely well with data teams & business leaders; if you want to be notified when that playbook is published, subscribe or leave a comment so it can be prioritized in a dedicated article.

2. Give ad-hoc work its own lane and capacity

For Kanban teams, create a dedicated swimlane or column for ad-hoc unplanned work with strict WIP limits. For Scrum teams, reserve a fixed slice of capacity each sprint (for example, 10–20%) specifically for ad-hoc and incidents; if your main problem is derailment of strategic work, be firm that once this buffer is full, new requests either wait for the next sprint or explicitly replace something of similar size.

3. Make trade-offs explicit with stakeholders

When a high-urgency request arrives mid-sprint, do not say yes by default; show the current sprint board and ask which existing ticket should move out if this new item truly has higher priority. Apply a simple rule: if something comes into the sprint mid-flight, something of similar size has to come out, preferably another

stakeholder ticket rather than foundational work like refactors or data-quality fixes, and confirm the decision in writing so the cost of interruption is visible and reviewable in retros later.

4. Turn patterns into improvements

Track basic attributes for every ad-hoc request (volume, size, source team, topic) so patterns emerge in a few weeks. Repeated questions become a backlog for self-service dashboards, documentation, or a better semantic layer, and if one department generates most interrupts, use sprint reviews or a stakeholder forum to align expectations, SLAs, or office-hours slots where their requests are triaged and handled.

For you as a CDO/VP, this is how you turn “data is a bottleneck into a transparent contract: which ad-hoc work fits, what it displaces, and how quickly different types of requests will be handled.

For the wider organization, this turns ad-hoc from a source of constant fire-drills into a predictable service: fewer Slack escalations, clearer expectations on turnaround, and less disruption to revenue-generating projects.

Are we better yet? Design your own way of working

Use a small set of metrics to see whether your way of working is improving predictability, quality, and trust, then tune your process based on what you see.

- **If you run mostly Kanban**, focus on flow and interruptions:

- Cycle time from “In Progress” to “Done” (separately for ad-hoc vs project work).
- WIP per person/column (In Progress/Review/Blocked).
- Unplanned work ratio: urgent vs planned tickets.
- **If you run mostly Scrum**, focus on predictability and quality:
 - Sprint predictability (finished/committed story points or ticket count).
 - Scope change inside the sprint: tickets added/removed after start.
 - Defects/incidents per sprint: production or serious data-quality issues.
- **If you run Scrumban/hybrid**, watch the boundary between roadmap and support:
 - Capacity split: % effort on planned work vs ad-hoc / incident lane.
 - Ad-hoc lead time: request creation to completion for the ad-hoc lane.

Across all modes, add one qualitative signal: a lightweight quarterly stakeholder pulse (for example, “Red/Amber/Green: how predictable and useful is our data support for you?”), and use all of these in retros to spot trends and pick one experiment per cycle.

For executives, these few flow and quality metrics plus a simple stakeholder pulse are enough for your leadership dashboard: are decisions happening faster, are incidents trending down, and are stakeholders reporting fewer nasty surprises from the data side?

Agile for data teams is not about copying textbook Scrum or Kanban.

it is about designing a way of working that fits your mix of BI, AE, DE, DS, and your stakeholders, starting with one or two changes (tighter standups, standardised intake, an ad-hoc lane) and seeing whether life actually gets better for both the team and the business.

If you are experimenting with your own flavour of Scrum, Kanban, Scrumban for data teams, leave a comment and share your setup—what is working, what is not, and which metrics or ceremonies you would like to see unpacked in future articles



13 Likes • 1 Restack

Discussion about this post

Comments Restacks



Write a comment...

1 more comment...

